

LA-5: More Data Wrangling (15 points)

Learning Outcomes

In this assignment, you will learn how to:

- Run and examine frequency distributions using the `freq()` function in the `summarytools` package.
- View descriptive statistics using the `descr()` function in the `summarytools` package.
- Recoding variables using the `case_when()` function.
- Create new variables from existing variables in a data frame using the `mutate()` function.

Tip

Read all the instructions carefully before starting the assignment.

Instructions

- 1) Install (`install.packages()`) and load (`library()`) the packages below. Then download `wind-turbines.csv` dataset from Canvas and read it in to R. You will also want to download the codebook for the data from Canvas.
 - `tidyverse`
 - `summarytools`
 - `rstatix`

Tip

Just like in previous assignments, read the data into an object using `read_csv()`. For example: `wind <- read_csv("wind-turbines.csv")`. Take a few minutes to look through the codebook so you know what each variable measures before you start answering questions below.

- 2) The dataset includes data about wind turbines in the US. Use the `freq()` function to examine the frequency distributions of the variables `Site.State` and `Year`. Use the frequency distributions to answer the following questions.
 - a) How many wind turbines are located in Utah?
 - b) How many wind turbines became operational in 2021?
 - c) What percentage of turbines became operational between 1983 and 1999 (include these years in the calculation)?

Note

The template for `freq()` is `freq(dataframe$variable)`. The output is a table listing each category (or, for a numeric variable like `Year`, each value), along with three useful columns: **Freq** (the count of cases with that value), **%** (the percentage of cases with that value), and a row at the bottom labeled **Total**, which gives the total number of cases in whatever data you ran `freq()` on.

To answer (c), you'll need to combine two skills:

- **Filtering on a range of values.** So far, you've filtered using `==` to match one exact value (e.g., `Show.Name == "Mamma Mia!"`). You can also use `>=` ("greater than or equal to") and `<=` ("less than or equal to") to filter for a range, and combine two conditions in one `filter()` step with `&` ("and"). For example, to keep only cases where `Year` is 1983 or later **and** 1999 or earlier:

```
df |>
  filter(variable >= value1 & variable <= value2)
```

- **Calculating a percentage.** Once you filter for `Year` between 1983 and 1999 and run `freq(Year)` on that filtered data, look at the **Total** row to see how many turbines became operational in that range. Then run `freq(Year)` on the full (unfiltered) dataset and look at its **Total** row to see the total number of turbines overall. Divide the first number by the second, and multiply by 100, to get a percentage.

- 3) Now, subset the data to answer the following question: How many turbines in Utah have been operational since 2010?

Tip

This requires combining two conditions with `&`, the same technique described above: one condition for the state and one for the year. "Since 2010" means 2010 and every year after, so use `>=`.

- 4) Examine the frequency distribution of `Turbine.Capacity` based on the subset in #3.
- a) How many turbines in this subset have a capacity of 2,300 kW?
 - b) What percentage of turbines in this subset have a capacity of 1,500 kW?
- 5) Often, we want to look at descriptive statistics of a quantitative variable. We can do this using the `descr()` function that is in the package `summarytools`. Examine the descriptive statistics for `Turbine.Total_Height` and answer the questions below. Include the units of measurement in your answers.
- a) What is the mean?
 - b) What is the median?
 - c) What is the maximum?

Note

The template for `descr()` is `descr(dataframe$variable)` (or `dataframe |> descr(variable)`, using the pipe). Unlike `freq()`, which counts cases in categories, `descr()` is used for **quantitative** (numeric) variables and produces a table of descriptive statistics, including the mean, median, minimum, maximum, and standard deviation. Find the row labeled with each statistic to answer the questions below, and check the codebook for the units (e.g., meters) that variable is measured in.

- 6) Next, we will learn how to use the function, `mutate()`. We use this function to create new variables (or "mutate" them) from existing ones. Let's create a new variable called `Turbine_Space` that is the mean of two existing variables, `Turbine.Hub_Height` and `Turbine.Rotor_Diameter`. Then, examine the descriptive statistics of the new variable, `Turbine_Space`, and answer the following questions:
- a) What is the mean of `Turbine_Space`?
 - b) What is the minimum value of `Turbine_Space`?
 - c) What is the maximum value of `Turbine_Space`?

Note

The general template for `mutate()` is:

```
df <- df |>
```

```
  mutate(new_variable_name = calculation_using_existing_variables)
```

Two things to keep in mind:

- **Save your changes.** Notice that the template starts with `df <-`. This reassigns the result back into your data frame. If you skip this and only run `df |> mutate(...)`, R will print the result to your Console but the new variable will not actually be saved — you won't be able to use it in later steps. This is the same idea as when you first read your data into an object with `bway <- read_csv(...)` or `wind <- read_csv(...)`.
- **R calculates row by row automatically.** To find the mean of two variables *for each row* (i.e., for each wind turbine), you don't need a special function — you can just add the two variables together and divide by 2, e.g., `(variable1 + variable2) / 2`. R automatically performs this calculation separately for every row in the dataset.

- 7) Sometimes, we may want to make continuous variables into categorical variables. For example, let's say we are interested in the number of turbines in a project, `Project.Number_Turbines`, but we want to categorize them into low vs. high numbers of turbines in projects.

We refer to this process as **recoding variables**. For this exercise, let's recode projects with 100 turbines or less into the **low** category and those that have more than 100 turbines into the **high** category.

Use the `mutate()` and `case_when()` functions to recode `Project.Number_Turbines` into a new variable, `Project.Number`, with only two categories, low (≤ 100) and high (> 100). Then, answer the following questions.

- a) How many projects are in the **low** category?
- b) How many projects are in the **high** category?

Note

The general template for combining `mutate()` and `case_when()` is:

```
df <- df |>
```

```
  mutate(new_variable_name = case_when(
    existing_variable <= value ~ "label1",
    existing_variable > value ~ "label2"
  ))
```

`case_when()` checks each row against your conditions, in order (using the same `==`, `>`, `<`, `>=`, and `<=` operators you've already used with `filter()`), and assigns the label after the `~` symbol to any row where that condition is `TRUE`. Just like with `mutate()` above, remember to reassign the result back to `wind` (i.e., start with `wind <- wind |> ...`) so that your new variable is saved and available for the questions below.

- 8) **BONUS:** In the previous step, you created a new variable called `Project.Number`. What is the mean `Project.Capacity` among projects in the (a) low and (b) high `Project.Number` categories?

Tip

One way to do this is to `filter()` for low and run `descr(Project.Capacity)`, then repeat for **high** — the same approach you've used throughout this assignment.

For a bonus challenge, look into the function `group_by()`. Adding `group_by(variable)` to your pipe *before* `descr()` will calculate the statistics separately for each category of `variable`, all in a single step: `df |> group_by(variable) |> descr(other_variable)`.

Submission

Submit your R script (named `LA-#_FirstName-LastName.R`) to Canvas.

Your R script should:

- 1) Include commands and functions that are necessary to address all the questions in the assignment.
- 2) Contain comments that answer the questions in the assignment.
- 3) Run in its entirety without errors.

To ensure that your R script runs without errors, you should:

- Save your script.
- Navigate back to Your Workspace on Posit Cloud.
- Reopen your project.
- Run the entire script line-by-line without editing it to ensure there are no errors.

! Important

These standards apply to all submissions in this course that require R scripts. You should follow these instructions for preparation, naming, and saving of your R script for all of your individual lab assignments.